

CASE STUDY 1 — Global ENERGY

Lead Data Engineer · Nov 2025 – Present · AWS Cloud · Energy Domain

STAR — PROJECT OVERVIEW

Situation

Global Energy operates a national smart metering, generation asset, and customer analytics estate spanning millions of domestic and commercial meters, generation telemetry feeds, and trading systems. Before this programme, meter, billing, and asset data arrived from a dozen heterogeneous source systems into a legacy on-premise estate with no consistent Bronze/Silver/Gold discipline — regulatory (Ofgem) reporting datasets were hand-assembled by analysts from a mix of extracts, and commercial teams routinely worked from data that was 24–48 hours stale.

Meter reconciliation failures were common: roughly 6–8% of daily meter reads failed downstream validation each month, usually discovered only after billing had already run, triggering costly billing corrections and customer complaints. Grid event and generation asset telemetry was similarly batch-only — operational teams had no way to see meter or grid anomalies until the next morning's report, by which point any settlement or safety issue had already propagated.

The business case for the modernisation programme was clear: reduce Ofgem regulatory reporting risk, bring meter and billing reconciliation error rates down, and give operational teams a genuinely current view of smart metering and grid events without destabilising the deterministic, auditable batch pipeline regulators require.

Task

As Lead Data Engineer, I was responsible for designing and delivering the AWS-native ingestion, transformation, and orchestration framework for smart meter, generation asset, energy trading, and customer consumption datasets; for introducing a lakehouse-grade Bronze/Silver/Gold structure on Apache Iceberg for Ofgem-compliant analytics; and for building the supplementary real-time streaming path for operational metering dashboards. I also owned production release governance for a 6-member data engineering team and mentored the team on PySpark, SQL, and Medallion architecture best practice.

Action

I designed the AWS Databricks ingestion frameworks in Python and PySpark that pull smart meter readings, generation asset telemetry, energy trading, and customer consumption data from heterogeneous source systems, landing raw data into Amazon S3 with explicit schema enforcement and partitioning by load date and source system.

I built the AWS Glue transformation jobs that apply cleansing, enrichment, reconciliation, and business-rule processing to meter and billing datasets as they move through Bronze, Silver, and Gold, specifically engineered for the regulatory reporting and commercial analytics consumers.

I implemented an Apache Iceberg-based lakehouse on AWS, choosing Iceberg over plain Parquet or Delta specifically for its native schema evolution, hidden partitioning, and time-travel — all of which are directly relevant to Ofgem audit and compliance requirements where I need to prove exactly what a dataset looked like at a point in time.

I engineered the real-time smart metering streaming pipeline using Apache Kafka and PySpark Structured Streaming with checkpointing and event-time processing, writing to Delta Lake as the streaming sink so meter readings and grid events reach operational dashboards continuously rather than overnight.

I built the Apache Airflow DAGs on Amazon MWAA that coordinate metering, billing, and asset analytics schedules — task dependencies, sensors, retry policies, SLA alerting, and parameterised Databricks job execution — and provisioned the entire supporting AWS estate (Databricks workspaces, S3, IAM, Glue, Secrets Manager, networking) using Terraform.

I owned production release governance for the team: code reviews, CI/CD deployment approvals via GitHub Actions, release validation, and rollback readiness across development and production environments.

Result

- Reduced meter reconciliation failure rate from 6–8% to under 1% by introducing schema-enforced Bronze landing plus automated schema evolution checks
- Cut the effective latency of operational meter and grid event visibility from next-day batch to under 2 minutes via the real-time streaming path
- Reduced Ofgem regulatory dataset preparation from a manual multi-day exercise to a fully automated same-day pipeline with reconciliation built in

- Achieved 99.9% pipeline SLA adherence on MWAA-orchestrated DAGs, down from frequent overnight failures on the legacy estate
- Standardised infrastructure provisioning via Terraform, cutting new-environment setup time from roughly 2 weeks to under 1 day
- Mentored a 6-engineer team to consistent PySpark/SQL/Medallion delivery standards, reducing code-review rework by an estimated 40%

ARCHITECTURE OVERVIEW

The platform runs two clearly separated tracks that share a common storage layer: a batch track carrying the full analytical and regulatory workload, and a supplementary real-time track for the specific low-latency use case below. They never share pipeline code — only the storage layer.

Batch Track

```

SOURCE SYSTEMS (meter head-end systems, generation SCADA extracts, billing DB, trading systems)
    |
    | [Python / PySpark ingestion frameworks]
    v
S3 RAW ZONE — Amazon S3 (raw/ prefix) — schema-enforced, partitioned by load_date
    |
    | AWS Glue Data Catalog (auto-crawled metadata)
    |
    | [AWS Lambda — Schema Evolution Check, triggered by EventBridge on S3 PutObject]
    |
    | [AWS Glue PySpark jobs — Silver cleansing, enrichment, reconciliation]
    v
S3 CURATED ZONE (Apache Iceberg tables) — Bronze → Silver → Gold, schema evolution +
upserts
    |
    | [dbt — Gold business models]
    v
AMAZON REDSHIFT — Star schema DWH for regulatory + commercial analytics
    |
    |
    v
CONSUMERS: Power BI · Ofgem regulatory submissions · Commercial analytics team
            Data Science (Athena over Iceberg tables directly)

ORCHESTRATION: Apache Airflow on Amazon MWAA — schedules & sequences every step above

```

Real-Time Track (separate track)

```

SMART METERS / GRID EVENT SOURCES
    |
    | [meter head-end event publisher]
    v
Apache Kafka (3-broker cluster, topics: raw → enriched → dlq)
    |
    | [PySpark Structured Streaming on Databricks — checkpointed, event-time
processing]
    |
    |-----|
    v               v
Delta Lake (S3 real-time zone — permanent record)  Amazon DynamoDB (NoSQL hot path)
(merged into next Iceberg batch run)                Single-digit ms reads
                                                    → Operational metering dashboard

```

SECTION A — BATCH PIPELINE (PRIMARY)

Start here in the interview. This is the core of the platform and should be walked through in full before any mention of real-time.

Step 1: Data Sources & Ingestion

Source systems fall into two ingestion patterns. Reference data — meter asset registry, tariff tables, generation asset master — uses a full load pattern (truncate + reload, validated by row count). Transactional data — meter reads, energy trading events, consumption records — uses incremental loading against a watermark held in a DynamoDB control table, because full loads of 50M+ daily meter reads would blow well past the sub-1-hour ingestion SLA.

All credentials for source connections are stored in AWS Secrets Manager — nothing is hardcoded, enforced by a pre-commit secret-scanning hook in the CI pipeline.

Ingestion tool: Python + PySpark ingestion frameworks on Databricks

Pattern: Full load (reference) + incremental watermark load (transactional)

Credential store: AWS Secrets Manager

Step 2: Bronze/Raw Layer

Meter, generation, trading, and consumption data lands in the S3 raw zone exactly as received — zero transformation — tagged with `ingested_at`, `source_system`, and `batch_run_id` audit columns and partitioned by `load_date` and `source_system`.

This is the single replay point: when a Silver reconciliation bug was found in month one, I reprocessed straight from Bronze with zero data loss, because Bronze had never been touched by the fix.

Format: Apache Iceberg tables over Parquet on S3

Partitioning: `load_date`, `source_system` (Iceberg hidden partitioning)

Retention: Raw zone retained 90 days before lifecycle transition to Glacier

Step 3: Schema Evolution (post-Bronze)

Immediately after Bronze landing completes, an AWS Lambda — triggered by an EventBridge rule on the S3 PutObject event — compares the newly landed schema against the expected schema held in the AWS Glue Data Catalog.

- Additive change (new column, e.g. a new tariff field): auto-approved, Glue Catalog schema updated, Iceberg mergeSchema enabled for the next Silver write, pipeline continues uninterrupted.
- Breaking change (dropped column or type change, e.g. a meter reading unit changing from kWh to Wh silently): the affected S3 partition is moved to a `/quarantine/` prefix, a `QUARANTINED` flag is written to the DynamoDB control table, and an SNS alert fires to the on-call Slack channel — Silver skips that source until a human clears the flag.

INTERVIEW TIP

How to answer "How did you handle unexpected schema changes?" — Describe the Lambda-on-EventBridge pattern, then give the concrete story: meter head-end vendors changed reading units without notice more than once; because the check auto-quarantines instead of silently propagating bad data, none of those changes reached the Gold layer or an Ofgem submission.

Step 4: Silver/Curated Transformation

AWS Glue PySpark jobs — triggered by Airflow only after the schema check passes — apply five transformation categories: type standardisation (timestamps to UTC ISO 8601), null handling (mandatory fields routed to a DLQ, never failing the whole job), business rules (tariff mapping, unit normalisation), deduplication (dedupe on `meter_id` + `reading_timestamp` keeping the latest), and derived metrics (daily consumption, generation utilisation %).

Silver never calls back to the source system — it only ever reads Bronze — which is what makes it safely re-runnable at any time.

- Records failing validation are written to a DLQ partition with a `rejection_reason` column rather than dropped — Airflow monitors DLQ volume and alerts above threshold.

Step 5: Gold/Serving Layer (dbt)

dbt models transform Silver Iceberg tables into business-ready fact and dimension tables in Amazon Redshift, structured as staging → intermediate → marts, with 150+ dbt tests running on every build.

Macros used: `generate_surrogate_key`, `safe_divide` (utilisation %), `date_spine` (settlement calendar), `create_schema_if_not_exists`

Step 6: Orchestration

Apache Airflow on Amazon MWAA is the scheduler and dependency orchestrator: it decides when each stage runs and enforces that Bronze completes and passes its schema check before Silver starts, and that Silver completes before the Gold dbt build begins.

AWS Glue is the execution engine that actually moves and transforms the bytes — it has no concept of cross-pipeline dependency. Airflow is the conductor; Glue is the orchestra playing the notes.

Orchestrator: Apache Airflow on Amazon MWAA

Execution engines: AWS Glue (Silver), Databricks jobs (ingestion), dbt (Gold)

Retry policy: 3 retries, exponential back-off (10/20/40 min), SLA alert to Slack then PagerDuty

INTERVIEW TIP

How to answer "What's the difference between ADF/Glue and Airflow?" — Glue is the workhorse: it has native connectors and moves and transforms the data. Airflow is the conductor: it schedules, sequences dependencies across completely different systems (Glue, Databricks, dbt), and is the single place SLA misses and failures are monitored and alerted on. Airflow DAGs are Python in Git — version-controlled and peer-reviewed; Glue job definitions on their own are not.

Step 7: Data Warehouse & Modelling

The Gold layer is a star schema in Redshift: FACT_METER_READS and FACT_GENERATION_OUTPUT as periodic-snapshot fact tables, joined to DIM_METER, DIM_CUSTOMER, DIM_TARIFF, and DIM_ASSET dimensions.

DIM_CUSTOMER is modelled as SCD Type 2 because tariff band and consumption profile at the time of a billing period matters for regulatory reconciliation — overwriting history would make it impossible to prove what tariff applied when a bill was generated.

- Surrogate keys generated via `dbt_utils.generate_surrogate_key` are used for every dimension join — never the natural meter_id or customer account number, which are strings and occasionally change format between source systems.
- Fact table rows are assigned the surrogate key of the dimension version that was current at the time of the event, so joins never need a date-range predicate at query time.

INTERVIEW TIP

How to answer "How do you handle slowly changing dimensions?" — Explain SCD2 on DIM_CUSTOMER: new row per tariff/profile change, eff_start_date/eff_end_date/is_current columns, and the fact table joined via the surrogate key assigned at load time — not a runtime BETWEEN date filter — because it is faster and avoids ambiguity when multiple changes land the same day.

Step 8: Data Quality Framework

Two layers: Great Expectations suites run on Bronze before any transformation (volume thresholds, mandatory-field null checks, meter reading regex format, referential integrity between meter and customer datasets); dbt tests run on Gold (uniqueness, not_null, relationships, accepted_values, plus custom range checks on consumption values).

Every load reconciles the Gold consumption total back to the source billing system control total — a variance above 0.01% blocks the Ofgem submission and pages the commercial reporting lead.

Step 9: Security & Data Governance

Access is controlled at three layers: S3/Iceberg storage via IAM and Lake Formation column-level permissions; the Glue/Databricks processing layer via least-privilege IAM roles per job; and Redshift via database users and groups with column exclusion for customer PII.

Customer PII (name, address, account number) is stored separately from consumption facts, encrypted at rest via AWS KMS customer-managed keys, in transit via TLS 1.2+, with all access logged via CloudTrail and retained for 7 years to satisfy Ofgem audit requirements.

INTERVIEW TIP

How to answer "How did you handle GDPR/regulatory compliance?" — Cover the three-layer access model, PII separated from fact data with KMS encryption, and the "right to erasure" pattern: deleting the customer PII row makes the associated consumption facts unidentifiable without touching historical regulatory reporting figures.

Step 10: How Data Reaches Consumers

Power BI and the commercial analytics team connect to Redshift via JDBC. Data science reads Iceberg tables directly through Athena, keeping heavy exploratory workloads off the shared warehouse. Ofgem regulatory submissions are produced by a dedicated Airflow task that reads Gold, formats to the required submission structure, runs the reconciliation check, and only then submits — this reconciliation step is what has kept the submission error rate at zero since go-live.

SECTION B — REAL-TIME PIPELINE (SUPPLEMENTARY)

Only raise this if asked. It is a parallel, smaller-scope track solving one specific latency problem — it does not replace the batch pipeline.

Why Real-Time is Needed

Smart meter and grid event data arriving only via overnight batch meant the operations team could not see a meter or grid anomaly — a suspected outage signature, a sudden generation drop — until the next morning's report, by which point a settlement or safety issue had already run its course. Operational dashboards needed meter and grid events within roughly 2 minutes of the physical event, not 12+ hours later.

Kafka Configuration

A 3-broker Kafka cluster with a replication factor of 3 handles the meter and grid event stream, structured as raw → enriched → dlq topics, message format Avro via Schema Registry so the producer enforces the event schema at write time.

Broker setup: 3-broker cluster, replication factor 3

Topic sizing: Peak ~450K events/hour ≈ 125 msgs/sec → 24 partitions for headroom

Format: Avro via Schema Registry

Consumer groups: One per downstream consumer (dashboard service, DynamoDB writer)

Stream Processing

PySpark Structured Streaming on Databricks reads the enriched topic with a 30-second processing trigger and a 10-minute watermark for late-arriving meter readings (some meters report over cellular links with variable latency). The stream is enriched with a broadcast join against the meter asset reference table, and writes to Delta Lake as the append-mode sink with checkpointing on S3 for exactly-once recovery after restart.

NoSQL Hot Path — Amazon DynamoDB

The operational dashboard needs single-digit-millisecond point lookups on "what is the latest reading for meter X" — a workload Redshift is not built for (Redshift is optimised for analytical scans over millions of rows, not per-key point reads at dashboard-refresh speed). DynamoDB is written to in parallel with the Delta Lake sink: partition key meter_id, sort key reading_timestamp, TTL of 90 days so old hot-path data expires automatically without manual cleanup.

INTERVIEW TIP

How to answer "Why NoSQL instead of your data warehouse?" — DynamoDB gives single-digit ms GetItem reads on a known key at effectively unlimited concurrency; Redshift is built for complex analytical SQL over large historical datasets and would add 100ms–5s of latency per lookup at a fraction of the concurrency DynamoDB supports. They serve different jobs — DynamoDB is the hot path for "now", Redshift is the system of record for analytics and regulatory reporting.

Connection Between Batch and Real-Time

Both tracks share the same S3 storage layer but write to separate prefixes. The daily batch pipeline reads both the standard raw prefix and the real-time zone for the same day, deduplicates on meter_id + reading_timestamp, and processes both through the same Silver/Gold path — so the Gold Iceberg tables always contain the complete, reconciled picture, while DynamoDB only ever serves the fast "right now" lookup.

SAMPLE DATA

FACT_METER_READS (periodic snapshot — one row per meter per settlement period)

reading_sk	meter_fk	tariff_fk	reading_ts	consumption_kwh	reading_type	settlement_period	validated_flag
5510042	MTR-884127	TRF-STD-01	2026-09-20T00:00	1.842	HALF_HOURLY	SP-01	TRUE
5510043	MTR-884127	TRF-STD-01	2026-09-20T00:30	1.910	HALF_HOURLY	SP-02	TRUE
5510091	MTR-221049	TRF-ECO7-02	2026-09-20T00:00	0.640	HALF_HOURLY	SP-01	TRUE
5510092	MTR-441882	TRF-STD-01	2026-09-20T00:00	4.220	HALF_HOURLY	SP-01	FALSE (pending)

reading_sk	meter_fk	tariff_fk	reading_ts	consumption_kwh	reading_type	settlement_period	validated_flag
							re-validation)
5510093	MTR-118844	TRF-BIZ-03	2026-09-20T00:00	18.750	HALF_HOURLY	SP-01	TRUE

DIM_CUSTOMER (SCD Type 2)

customer_sk	customer_id	tariff_band	region	eff_start_date	eff_end_date	is_current
30041	CUST-Global-448821	STANDARD	SOUTH_EAST	2024-01-01	2026-03-14	FALSE (moved to Eco7)
30089	CUST-Global-448821	ECO7	SOUTH_EAST	2026-03-15	9999-12-31	TRUE (current version)
30042	CUST-Global-221049	ECO7	MIDLANDS	2023-06-01	9999-12-31	TRUE
30043	CUST-Global-118844	BUSINESS_TIER1	LONDON	2022-01-01	9999-12-31	TRUE

Real-Time Event — Field Table

Kafka/Kafka topic: Global.smartmeter.readings.raw

Field	Type	Example
meter_id	string	MTR-884127
reading_ts	timestamp	2026-09-22T14:31:07.000Z
consumption_kwh	decimal	1.842
reading_type	string	HALF_HOURLY
signal_source	string	CELLULAR
grid_event_flag	string	NONE (NONE/VOLTAGE_DIP/OUTAGE_SUSPECTED)
region	string	SOUTH_EAST
device_battery_pct	integer	78